Reviewer Pack — Tropical Representation Theory and AC0 Parity Circuit Comple…

Entry 3f194adec2d6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 17:35:27 UTC

Tropical Representation Theory and AC0 Parity Circuit Complexity

Entry ID: `3f194adec2d6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 17:35:27 UTC

1. Conjecture

Field A (mathematical branch): Tropical Representation Theory **Field B** (complexity object): AC0 Parity Circuit Complexity

Statement:

> For any AC0 parity circuit C with size s, there exists a vector space V with dimension d ≥ log_2(s) such that the tropicalized representation of the output vector of C is isomorphic to the tensor product of a fixed representation space T and the dual of an irreducible representation space W.

Rationale (proposer's reasoning):

> Tropical representation theory provides a framework for studying linear algebra over the tropical semiring, which may reveal hidden structures in AC0 circuits that are not apparent when considering their traditional Boolean representations. This conjecture suggests that the complexity of computing parity can be characterized by the dimensions and structure of these tropical vector spaces.

Taxonomy category: `AC0_PARITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7cf492449ee767c0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For AC0 parity circuits, if dimension d of tropical vector space V satisfies d ≥ log_2(s) AND for all circuits s ≤ 40, there exists an irreducible representation space W with |W| = 1, then the conjecture is supported. If any circuit has d < log_2(s) OR |W| > 1, then the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical representation theory AND AC0 parity circuit complexity` - `AC0 circuits AND tropicalized representations dimension >= log2(size)` - `irreducible representations tensor product dual in AC0 parity circuits with size <= s`

Top relevant hits considered: - [s2:1608.04355] Polynomial Representations of Threshold Functions and Algorithmic Applications

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def log2(x):
        if x <= 0:
            return float('-inf')
        return math.log2(x)

    def max_element(lst):
        return max(lst, default=float('-inf'))

    def tropical_add(a, b):
        return max(a, b)

    def tropical_mul(a, b):
        return a + b

    def tropical_exp(a, b):
        if a == float('-inf'):
            return b
        if b == float('-inf'):
            return a
        return max(a, b)
```

```
def tropical_neg(a):
    return -a

def tropical_inv(a):
    if a == 0:
        return float('inf')
    return 1 / a

def tropical_zero():
    return float('-inf')

def tropical_one():
    return 0

def tropical_is_zero(a):
    return a == float('-inf')

def tropical_is_one(a):
    return a == 0

def tropical_eq(a, b):
    return a == b

def tropical_neq(a, b):
    return a != b

def tropical_lt(a, b):
    return a < b

def tropical_leq(a, b):
    return a <= b

def tropical_gt(a, b):
    return a > b

def tropical_geq(a, b):
    return a >= b

def tropical_add_list(lst):
    if not lst:
        return float('-inf')
    result = lst[0]
    for x in lst[1:]:
        result = tropical_add(result, x)
    return result

def tropical_mul_list(lst):
    if not lst:
        return 0
    result = lst[0]
    for x in lst[1:]:
        result = tropical_mul(result, x)
    return result

def tropical_exp_list(lst):
    if not lst:
```

```
        return float('-inf')
    result = lst[0]
    for x in lst[1:]:
        result = tropical_exp(result, x)
    return result

def tropical_neg_list(lst):
    return [tropical_neg(x) for x in lst]

def tropical_inv_list(lst):
    return [tropical_inv(x) for x in lst]

def tropical_zero_list(n):
    return [tropical_zero() for _ in range(n)]

def tropical_one_list(n):
    return [tropical_one() for _ in range(n)]

def tropical_is_zero_list(lst):
    return all(tropical_is_zero(x) for x in lst)

def tropical_is_one_list(lst):
    return all(tropical_is_one(x) for x in lst)

def tropical_eq_list(lst1, lst2):
    return all(tropical_eq(x, y) for x, y in zip(lst1, lst2))

def tropical_neq_list(lst1, lst2):
    return any(tropical_neq(x, y) for x, y in zip(lst1, lst2))

def tropical_lt_list(lst1, lst2):
    return all(tropical_lt(x, y) for x, y in zip(lst1, lst2))

def tropical_leq_list(lst1, lst2):
    return all(tropical_leq(x, y) for x, y in zip(lst1, lst2))

def tropical_gt_list(lst1, lst2):
    return all(tropical_gt(x, y) for x, y in zip(lst1, lst2))

def tropical_geq_list(lst1, lst2):
    return all(tropical_geq(x, y) for x, y in zip(lst1, lst2))

def tropical_add_matrix(A, B):
    return [[tropical_add(a, b) for a, b in zip(row_a, row_b)] for row_a, row_b in zip(A, B)]

def tropical_mul_matrix(A, B):
    result = []
    for i in range(len(A)):
        row = [tropical_add_list([tropical_mul(a, b) for a, b in zip(col_A, row_B)]) for col_A
        result.append(row)
    return result

def tropical_exp_matrix(A, B):
    result = []
    for i in range(len(A)):
        row = [tropical_add_list([tropical_exp(a, b) for a, b in zip(col_A, row_B)]) for col_A
```

```
        result.append(row)
    return result

def tropical_neg_matrix(A):
    return [[tropical_neg(x) for x in row] for row in A]

def tropical_inv_matrix(A):
    n = len(A)
    I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    M = [row[:] + col for row, col in zip(A, I)]
    for k in range(n):
        pivot = M[k][k]
        if pivot == float('-inf'):
            return None
        for j in range(k, n * 2):
            M[k][j] /= pivot
        for i in range(n):
            if i != k:
                factor = M[i][k]
                for j in range(k, n * 2):
                    M[i][j] -= factor * M[k][j]
    return [row[n:] for row in M]

def tropical_zero_matrix(n):
    return [[tropical_zero() for _ in range(n)] for _ in range(n)]

def tropical_one_matrix(n):
    return [[1 if i == j else 0 for j in range(n)] for i in range(n)]

def tropical_is_zero_matrix(A):
    return all(tropical_is_zero(x) for row in A for x in row)

def tropical_is_one_matrix(A):
    return all(tropical_is_one(x) for row in A for x in row)

def tropical_eq_matrix(A, B):
    return all(all(tropical_eq(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(A,

def tropical_neq_matrix(A, B):
    return any(any(tropical_neq(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(A

def tropical_lt_matrix(A, B):
    return all(all(tropical_lt(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(A,

def tropical_leq_matrix(A, B):
    return all(all(tropical_leq(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(A

def tropical_gt_matrix(A, B):
    return all(all(tropical_gt(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(A,

def tropical_geq_matrix(A, B):
    return all(all(tropical_geq(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(A

def tropical_add_list_of_lists(lst):
    if not lst:
        return []
```

```
    result = [lst[0][i] for i in range(len(lst[0]))]
    for sublist in lst[1:]:
        result = [tropical_add(a, b) for a, b in zip(result, sublist)]
    return result

def tropical_mul_list_of_lists(lst):
    if not lst:
        return []
    result = [lst[0][i] for i in range(len(lst[0]))]
    for sublist in lst[1:]:
        result = [tropical_mul(a, b) for a, b in zip(result, sublist)]
    return result

def tropical_exp_list_of_lists(lst):
    if not lst:
        return []
    result = [lst[0][i] for i in range(len(lst[0]))]
    for sublist in lst[1:]:
        result = [tropical_exp(a, b) for a, b in zip(result, sublist)]
    return result

def tropical_neg_list_of_lists(lst):
    return [[tropical_neg(x) for x in row] for row in lst]

def tropical_inv_list_of_lists(lst):
    return [[tropical_inv(x) for x in row] for row in lst]

def tropical_zero_list_of_lists(n, m):
    return [[tropical_zero() for _ in range(m)] for _ in range(n)]

def tropical_one_list_of_lists(n, m):
    return [[1 if i == j else 0 for j in range(m)] for i in range(n)]

def tropical_is_zero_list_of_lists(lst):
    return all(tropical_is_zero(x) for row in lst for x in row)

def tropical_is_one_list_of_lists(lst):
    return all(tropical_is_one(x) for row in lst for x in row)

def tropical_eq_list_of_lists(lst1, lst2):
    return all(all(tropical_eq(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(ls

def tropical_neq_list_of_lists(lst1, lst2):
    return any(any(tropical_neq(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(l

def tropical_lt_list_of_lists(lst1, lst2):
    return all(all(tropical_lt(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(ls

def tropical_leq_list_of_lists(lst1, lst2):
    return all(all(tropical_leq(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(l

def tropical_gt_list_of_lists(lst1, lst2):
    return all(all(tropical_gt(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(ls

def tropical_geq_list_of_lists(lst1, lst2):
    return all(all(tropical_geq(a, b) for a, b in zip(row_a, row_b)) for row_a, row_b in zip(l
```

```python
def tropical_add_matrix_of_matrices(A, B):
    result = []
    for i in range(len(A)):
        row = [tropical_add_list([tropical_add(a, b) for a, b in zip(col_A, col_B)]) for col_A
        result.append(row)
    return result

def tropical_mul_matrix_of_matrices(A, B):
    result = []
    for i in range(len(A)):
        row = [tropical_add_list([tropical_mul(a, b) for a, b in zip(col_A, col_B)]) for col_A
        result.append(row)
    return result

def tropical_exp_matrix_of_matrices(A, B):
    result = []
    for i in range(len(A)):
        row = [tropical_add_list([tropical_exp(a, b) for a, b in zip(col_A, col_B)]) for col_A
        result.append(row)
    return result

def tropical_neg_matrix_of_matrices(A):
    return [[tropical_neg_list(row) for row in matrix] for matrix in A]

def tropical_inv_matrix_of_matrices(A):
    return [[tropical_inv_list(row) for row in matrix] for matrix in A]

def tropical_zero_matrix_of_matrices(n, m):
    return [[[tropical_zero() for _ in range(m)] for _ in range(m)] for _ in range(n)]

def tropical_one_matrix_of_matrices(n, m):
    return [[[1 if i == j else 0 for j in range(m)] for _ in range(m)] for _ in range(n)]

def tropical_is_zero_matrix_of_matrices(A):
    return all(tropical_is_zero_list_of_lists(matrix) for matrix in A)

def tropical_is_one_matrix_of_matrices(A):
    return all(tropical_is_one_list_of_lists(matrix) for matrix in A)

def tropical_eq_matrix_of_matrices(A, B):
    return all(all(tropical_eq_list_of_lists(matrix_A, matrix_B) for matrix_A, matrix_B in zip

def tropical_neq_matrix_of_matrices(A, B):
    return any(any(tropical_neq_list_of_lists(matrix_A, matrix_B) for matrix_A, matrix_B in zi

def tropical_lt_matrix_of_matrices(A, B):
    return all(all(tropical_lt_list_of_lists(matrix_A, matrix_B) for matrix_A, matrix_B in zip

def tropical_leq_matrix_of_matrices(A, B):
    return all(all(tropical_leq_list_of_lists(matrix_A, matrix_B) for matrix_A, matrix_B in zi

def tropical_gt_matrix_of_matrices(A, B):
    return all(all(tropical_gt_list_of_lists(matrix_A, matrix_B) for matrix_A, matrix_B in zip

def tropical_geq_matrix_of_matrices(A, B):
```

```
        return all(all(tropical_geq_list_of_lists(matrix_A, matrix_B) for matrix_A, matrix_B in zi

    def tropical_add_list_of_lists_of_lists(lst):
        if not lst:
            return []
        result = [lst[0][i] for i in range(len(lst[0]))]
        for sublist in lst[1:]:
            result = [tropical_add_list(a, b) for a, b in zip(result, sublist)]
        return result

    def tropical_mul_list_of_lists_of_lists(lst):
        if not lst:
            return []
        result = [lst[0][i] for i in range(len(lst[0]))]
        for sublist in lst[1:]:
            result = [tropical_mul_list(a, b) for a, b in zip(result, sublist)]
        return result

    def tropical_exp_list_of_lists_of_lists(lst):
        if not lst:
            return []
        result = [lst[0][i] for i in range(len(lst[0]))]
        for sublist in lst[1:]:
            result = [tropical_exp_list(a, b) for a, b in zip(result, sublist)]
        return result

    def tropical_neg_list_of_lists_of_lists(lst):
        return [[tropical_neg_list(row) for row in matrix] for matrix in lst]

    def tropical_inv_list_of_lists_of_lists(lst):
        return [[tropical_inv_list(row) for row in matrix] for matrix in lst]

    def tropical_zero_list_of_lists_of_lists(n, m, p):
        return [[[tropical_zero() for _ in range(p)] for _ in range(m)] for _ in range(n)]

    def tropical_one_list_of_lists_of_lists(n, m, p):
        return [[[1 if i == j else 0 for j in range(p)] for _ in range(m)] for _ in range(n)]

    def tropical_is_zero_list_of_lists_of_lists(lst):
        return all(tropical_is_zero_matrix_of_matrices(matrix) for matrix in lst)

    def tropical_is_one_list_of_lists_of_lists(lst):
        return all(tropical_is_one_matrix_of_matrices(matrix) for matrix in lst)
# ... [truncated, full source in replay tarball]
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b080894.py", line 80, in <module>
    result = run_trial(seed)
```

```
            ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b080894.py", line 53, in run_trial
    tropical_output = [max(output_vector[i], T[i][j]) for j in range(d)]
                                           ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed due to an undefined variable error, preventing the production of data necessary to evaluate the conjecture. | next: Debug and correct the error in the test code to proceed with the evaluation.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13107
2	preregistration	ollama_remote	glm4:latest	0	0	9507
3	novelty	ollama_remote	glm4:latest	0	0	9761
4	novelty	ollama_remote	glm4:latest	0	0	8877
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12045
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14640
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10061
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	86255
9	judge	ollama_remote	glm4:latest	0	0	11465

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 175716 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3f194adec2d6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3f194adec2d6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3f194adec2d6.tar.gz` (if generated)